

Code Review #2 — recapitulare-vacanta

Stadiu: 1a  (două variante) · 1b  rezolvat în `solutie2` (cu un bug de limită, vezi  1). Ai citit enunțul corect — cifra maximă se ia din **tot** vectorul, iar excluderea min/max se aplică doar la numărare. Exact așa.

Ce ai aplicat din review-ul anterior

Punct	Stadiu
<code>v[300]</code> conform enunțului	 aplicat
Organizare header	 interpretare proprie: <code>fisier.h</code> → <code>functii.h</code> — merge și așa
Mesaje de commit descriptive	 mult mai bine decât „upd”
 off-by-one la zerorizarea lui <code>f</code>	 încă acolo (<code>functii.h:67</code>)
 <code>valMinima</code> fără <code>return</code> pe toate căile	 încă acolo (<code>functii.h:77-86</code>)
 numere negative + vector de frecvență	 nediscutat
Verificare deschidere <code>data.txt</code>	 nefăcut
Spații în mesaje afișate	 nefăcut
<code>.gitignore</code> pentru <code>bin/</code> și <code>obj/</code>	 nefăcut

Bug-urile  vechi rămân valabile — `solutie12` nu mai e apelată din `main`, dar codul greșit rămâne în `functii.h` și te va mușca la 1c, unde vei refolosi frecvența. Le găsești detaliate în review-ul #1; aici mă concentrez pe codul nou.

Bug-uri critice (rezolvă întâi)

1. `if(n>13)` respinge exact cazul `n=13` (`solutii.h:34`)

Al 13-lea element este `v[12]`, care există de la `n >= 13`. Condiția ta `n > 13` înseamnă `n >= 14`, deci pentru un vector cu exact 13 elemente afișezi „lungime necorespunzătoare” deși al 13-lea element există.

Fix:

```
if(n >= 13)
```

Verifică: pune 13 pe prima linie din `data.txt` (și 13 numere) — programul trebuie să afișeze contorul, nu mesajul de eroare.

2. `frecventaCifraVector` nu inițializează `f` (`functii.h:88`)

Funcția doar incrementează — merge acum pentru că în `solutie2` ai declarat `int f[10]{} (zero-inițializat)`. Dar contractul e fragil: cine apelează funcția a doua oară, sau fără `{}`, obține contoare acumulate sau gunoi. Compară cu `frecventaNumarVector`, care își face singură zerorizarea — cele două funcții-surori au contracte diferite și asta produce bug-uri exact când te grăbești.

Fix: adaugă zerorizarea la începutul funcției:

```
void frecventaCifraVector(int v[], int n, int f[])
{
    for(int i = 0; i < 10; i++)
    {
        f[i] = 0;
    }
    ...
}
```

Verifică: apelează funcția de două ori la rând pe același `f` — rezultatul trebuie să fie identic.

🟡 Issue-uri minore (când ai timp)

1. `solutie2` strică vectorul — `v[pozMa]=0; v[pozMi]=0;` e un truc ingenios pentru excludere (0 nu produce cifre în `while(aux>0)`), dar modifică vectorul definitiv. Orice calcul de după folosește date false. Pentru 1f/1g vei avea nevoie de vectorul intact — obișnuiește-te să lucrezi pe o copie sau să sari pozițiile în buclă:

```
if(i == pozMa || i == pozMi) continue;
```

2. Elementul 0 nu are cifre în `frecventaCifraVector` — `while(aux>0)` nu rulează pentru `aux=0`, dar numărul 0 are cifra 0. Dacă în vector există un 0 „adevărat” (nu pus de tine), cifra lui se pierde.

3. **Excluderea min/max cu duplicate** — dacă minimul apare de 3 ori, tu excluzi doar apariția de la `pozMin`. Enunțul e ambiguu aici; de discutat la sesiune ce interpretare alegem.
4. **Ordinea afișării în `solutie2`** — afișezi întâi contorul din al 13-lea element, apoi cifra maximă. Cititorul vede „Apare de 2 ori” fără să știe încă ce apare. Afișează întâi `cifMax`.
5. **Typo:** „coresponzutaoare” → „corespunzatoare” (`solutii.h:38`).
6. **`valMax` e definită dar neapelată** — ai folosit `pozMax` în loc. E ok să o păstrezi pentru mai târziu, dar știi de ce ai două funcții care se suprapun (`valMax(v,n) == v[pozMax(v,n)]`)? Poți rescrie una prin cealaltă.
7. **Mesajul de commit** — „rezolvare exercitiul 2” e de fapt problema 1, punctul b). Când mesajul nu corespunde conținutului, istoricul te minte peste o lună.
8. **Rămase din review-ul #1:** verificarea `data.txt`, spațiile din mesaje (`"...este"<< mn`), `.gitignore` + `git rm -r --cached bin obj`.

● Pattern-uri corecte (continuă așa)

- **Citirea atentă a enunțului la 1b** — cifra maximă din tot vectorul, excludere doar la numărare. Mulți greșesc exact aici.
- **Trucul `v[poz]=0`** — soluție creativă la excludere; acum rafinează-o (vezi ● 1).
- **`int f[10]{}`** — zero-inițializare modernă C++11, corect folosită.
- **Descompunerea în cifre** cu `%10` și `/10` e curată și consecventă în toate cele trei funcții (`cifMaxNumar`, `contorCif`, `frecventaCifraVector`).
- **Refolosirea funcțiilor** — `cifMaxVector` apelează `cifMaxNumar` în loc să dubleze logica.
- **`pozMin / pozMax`** — comparație prin `v[p]`, fără variabile auxiliare inutile.
- **Fix-uri aplicate din review** + redenumirea `functii.h` arată că gândești organizarea, nu doar execuți.

Pașii următori

Pas	Cerință	Ai deja	Îți lipsește
● din ambele review-uri	—	—	fix-urile de mai sus + cele 3 din review #1
1c	frecvența minimului între pozițiile pmin și pmax	<code>pozMin</code> , <code>pozMax</code> , <code>valMin</code> , <code>nrAp</code>	o numărare doar pe intervalul <code>[pmin, pmax]</code>
1d	toate elementele conțin cifra min sau max?	<code>contorCif</code>	cifra <i>minimă</i> (oglanda lui <code>cifMaxNumar</code>)

Pas	Cerință	Ai deja	Îți lipsește
1e	primul minim absolut	—	citește exemplul din culegere de 2 ori
1f, 1g	interschimbări min/max	<code>pozMin</code> , <code>pozMax</code> , <code>nrAp</code>	vector nemodificat — vezi 🟡 1

Sinteză

Criteriu	Review #1	Acum
Progres culegere	1a	1a + 1b
Bug-uri critice deschise	3	5 (3 vechi + 2 noi)
Igienă git	✗	⚠️ mesaje bune, lipsește <code>.gitignore</code>
Robustețe (fișier, n mic, negative)	✗	✗ neschimbat
Aplicare feedback	—	3 din 9 puncte

Ritmul e bun — o problemă pe zi cu două abordări la prima. Prioritatea acum: **închide bug-urile** 🟡
înainte de 1c, pentru că 1c refolosește exact funcțiile în care stau ele.